



DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

Ingeniería de Software

Análisis y Diseño de Software - NRC 30723

TransControl

Patrones de diseño

Grupo

Egas Steven

Obando Alejandro

Quimbiulco Juan

Instructor: Ing. Jenny Alexandra Ruiz Robalino

Fecha: 17/06/2026

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: <u>Ciencias de la Computación</u>
	Carrera: <u>Ingeniería de Software</u> Asignatura: <u>Análisis y Diseño de Software</u> NRC: <u>30723</u>
	Apellidos y nombres del estudiante: <u>Egas Jácome Steven David</u> <u>Obando Zapata Alejandro Xavier</u> <u>Qumbiulco Carrion Juan Diego</u>

Diseño e implementación de la arquitectura de tres capas para la Gestión de Transportistas

Diseñar e implementar una arquitectura de software basada en el modelo de Tres Capas para el módulo Gestión de Transportistas del sistema TransControl. El objetivo principal es lograr una separación clara y escalable de responsabilidades dividiendo el sistema en:

1. **Capa de Datos (Acceso y persistencia)**
2. **Capa de Lógica de Negocio (Reglas y servicios)**
3. **Capa de Presentación (Interfaces de usuario y controladores)**

Esta arquitectura permite soportar de forma estructurada las operaciones de Crear, Consultar, Actualizar y Eliminar (CRUD) sobre los transportistas.

1.1 Atributos Considerados (Modelo Transportista)

Según los requerimientos, la entidad principal gestiona los siguientes datos:

- Cédula
- Nombres y Apellidos (Nombres Completos)
- Correo Electrónico
- Dirección
- Teléfono
- Estado (Activo / Inactivo)

2. Insumos Obligatorios

Para el correcto desarrollo de esta arquitectura se tomaron como base los siguientes documentos y artefactos:

- ERS/SRS del proyecto TransControl: Especificaciones de requisitos del sistema.
- Libros de NotebookLM de Análisis y Diseño del proyecto

Casos de Uso:

- Crear Transportista
- Editar Transportista
- Eliminar Transportista
- Importar Documentación

Modelo de Dominio: Entidad Transportista.

3. Desarrollo de la Actividad

3.1 Análisis de Requisitos

Se identificó la necesidad de construir un sistema altamente modular que permita desacoplar la interfaz de usuario de las reglas de negocio y el almacenamiento de datos. Para esto, se implementó Node.js con TypeScript en el Backend y React en el Frontend.

3.2 Identificación de Clases y Entidades Principales

- Alineado con el paradigma orientado a objetos y la inyección de dependencias, se crearon las siguientes clases e interfaces:
- Transportista (Entidad de Dominio): Define el contrato y los atributos básicos del transportista.
- ITransportistaRepository (Interfaz): Abstracción del repositorio de datos.
- JsonTransportistaAdapter (Implementación de Datos): Clase concreta que implementa el repositorio y maneja el acceso al almacenamiento local (JSON en este caso).
- TransportistaService (Servicio de Negocio): Clase que contiene la lógica central del CRUD, encargada de validar antes de persistir o devolver datos.
- TransportistaController (Controlador de Presentación): Recibe las peticiones HTTP (Frontend), interactúa con el servicio y devuelve respuestas.
- TransportistaList / CrearTransportista (Formularios UI): Componentes visuales en React (Frontend) que representan la capa más externa de la presentación.

3.3 Diseño de la Arquitectura en Tres Capas

La solución fue dividida estructuralmente en el código fuente de la siguiente manera:

A. Capa de Presentación (Frontend y Controladores)

Se encarga de la interacción directa con el usuario final.

Frontend (React): Ubicado en `frontend/src/pages/`. Muestra las listas y formularios.

Backend (Controladores HTTP): Ubicado en `backend/src/presentation/controllers/`. Recibe peticiones desde el cliente web y traduce estas solicitudes hacia la lógica de negocio.

B. Capa de Lógica de Negocio (Servicios y Dominio)

Es el núcleo de la aplicación, contiene las reglas de operación.

Servicios: Ubicado en `backend/src/business/services/TransportistaService.ts`. Orquesta el flujo de datos.

Dominio: Ubicado en `backend/src/domain/entities/Transportista.ts`. Establece los modelos puros independientes del entorno.

C. Capa de Datos (Repositorios y Adaptadores)

Se encarga de comunicarse con la base de datos o almacenamiento.

Repositorios: Ubicado en `backend/src/data/adapters/`. Permite aislar la capa de negocio de los detalles de implementación de la base de datos, facilitando futuras migraciones a SQL o NoSQL.

3.4 Implementación y Validaciones

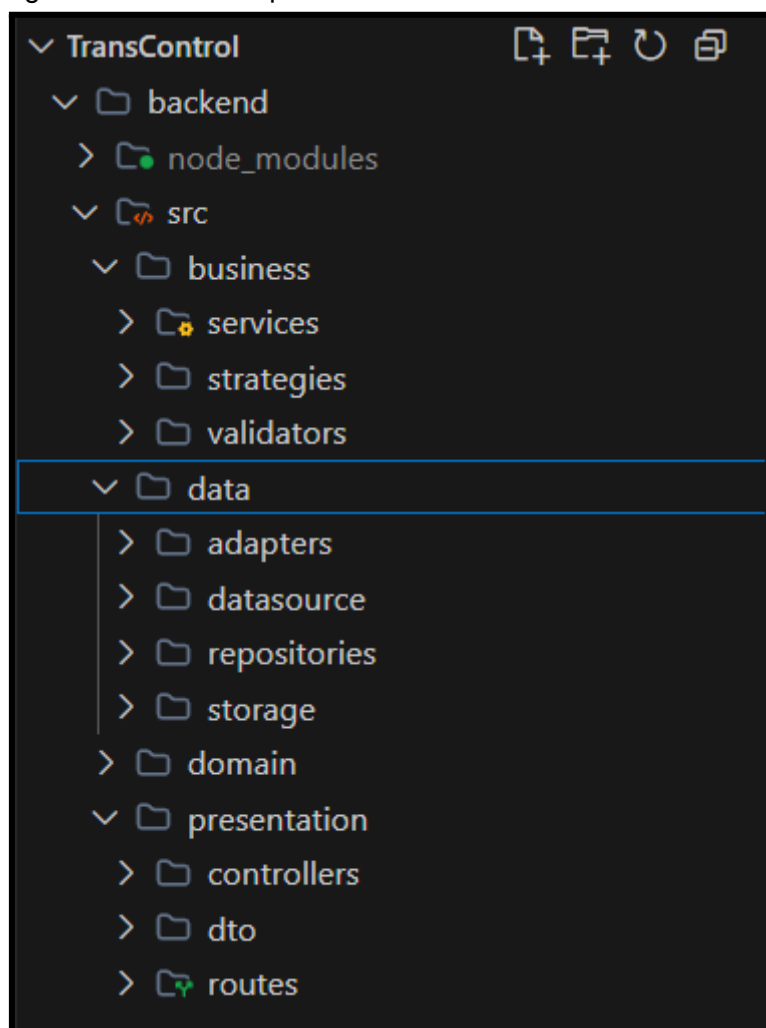
Operaciones CRUD: Se desarrollaron completamente los métodos `create`, `getAll`, `getById`, `update` y `delete`.

Validación de Datos: Se asegura que los atributos obligatorios (como cédula, nombres y correo) no estén vacíos en la capa de negocio antes de proceder a la creación, delegando en herramientas de validación (schemas) los formatos correctos (ej. validación del correo).

Documentación: El código cuenta con la clara separación de responsabilidades exigida, donde un cambio en la interfaz gráfica no altera el servicio, y un cambio de persistencia (Base de Datos) no altera la lógica.

Productos verificables

- Proyecto base organizado en tres capas.



- Diagrama o esquema de arquitectura aplicada

```
@startuml
    Arquitectura_TransControl
    title TransControl - Arquitectura de Tres Capas y Patrones de Diseño
    left to right direction
    skinparam shadowing false
    skinparam packageStyle rectangle
    skinparam rectangleRoundCorner 10
    skinparam componentStyle rectangle
    skinparam linetype ortho
    actor "Usuario" as Usuario
    package "Capa de Presentación" as Presentacion #DCEEFF {
        rectangle "Dashboard" as Dashboard
        rectangle "ViajesList\n(React)" as ViajesScreen
        rectangle "TransportistaList\n(React)" as TransportistasScreen
        rectangle "DocumentacionList\n(React)" as DocumentosScreen
        rectangle "TransportistaController\n(Node.js API)" as TransController
        rectangle "ViajeController\n(Node.js API)" as ViajeController}
    package "Capa de Negocio" as Negocio #E1F5E6 {
        rectangle "TransportistaService" as TransService
        rectangle "ViajeService" as ViajeService
        rectangle "ViajeSubject\n<<Observer Subject>>" as Subject
        rectangle "TravelObserver\n<<Observer>>" as Observer
        rectangle "CoordinadorObserver" as CoordObs
        rectangle "SecretariaObserver" as SecObs
        rectangle "TransportistaObserver" as TransObs
        rectangle "RutaCalculadora\n<<Context>>" as RutaCalculadora
        rectangle "RouteStrategy\n<<Strategy>>" as RouteStrategy
        rectangle "RutaMasRapidaStrategy" as Rapida
        rectangle "RutaMasSeguraStrategy" as Segura
        rectangle "RutaMenorDistanciaStrategy" as Distancia}
    package "Capa de Datos" as Datos #F1E3FA {
        rectangle "ITransportistaRepository\n<<Target>>" as TransportistaRepo
        rectangle "IViajeRepository\n<<Target>>" as ViajeRepo
        rectangle "JsonTransportistaAdapter\n<<Adapter>>" as TransAdapter
        rectangle "JsonViajeAdapter\n<<Adapter>>" as ViajeAdapter
        database "transportistas.json / viajes.json\nFuente de datos (Archivo JSON)" as FuenteDatos #FFFFFF}
    package "Modelo / Entidades" as Modelo #FFF4CC {
        rectangle "Viaje" as Viaje
        rectangle "Ruta" as Ruta
        rectangle "Transportista" as Transportista
        rectangle "Documento" as Documento}
```

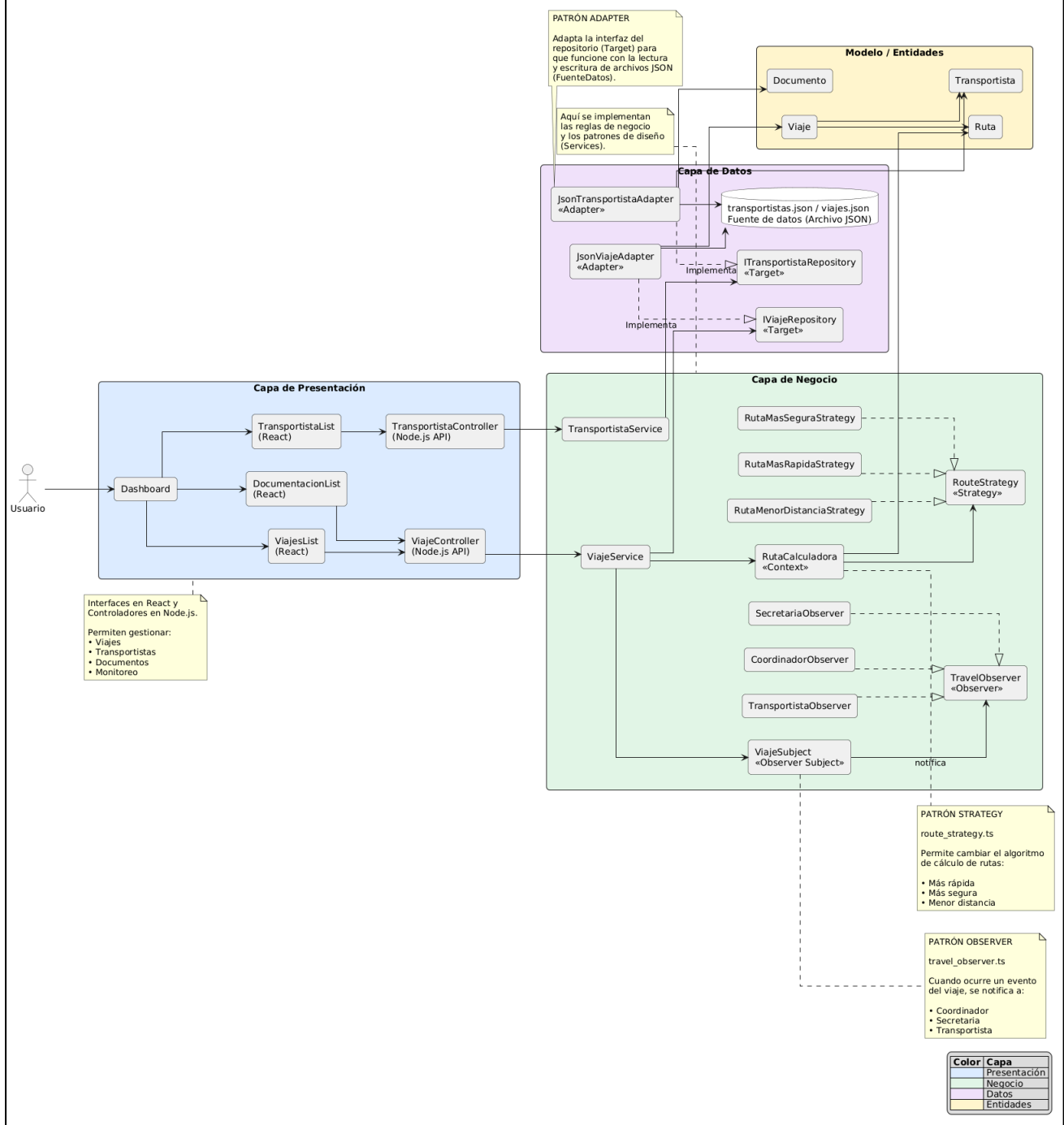
```
Usuario --> Dashboard
Dashboard --> ViajesScreen
Dashboard --> TransportistasScreen
Dashboard --> DocumentosScreen
TransportistasScreen --> TransController
ViajesScreen --> ViajeController
DocumentosScreen --> ViajeController
TransController --> TransService
ViajeController --> ViajeService
ViajeService --> ViajeRepo
TransService --> TransportistaRepo
TransAdapter ..|> TransportistaRepo : Implementa
ViajeAdapter ..|> ViajeRepo : Implementa
TransAdapter --> FuenteDatos
ViajeAdapter --> FuenteDatos
ViajeService --> Subject
TransAdapter --> Transportista
TransAdapter --> Documento
ViajeAdapter --> Viaje
Subject --> Observer : notifica
CoordObs ..|> Observer
SecObs ..|> Observer
TransObs ..|> Observer
ViajeService --> RutaCalculadora
RutaCalculadora --> RouteStrategy
Rapida ..|> RouteStrategy
Segura ..|> RouteStrategy
Distancia ..|> RouteStrategy
RutaCalculadora --> Ruta
Viaje --> Ruta
Viaje --> Transportista
note top of Presentacion
Interfaces en React y
Controladores en Node.js.
Permiten gestionar:
• Viajes
• Transportistas
• Documentos
• Monitoreo
end note
note top of Negocio
Aquí se implementan
las reglas de negocio
```

```

y los patrones de diseño
(Services).
end note
note right of Subject
PATRÓN OBSERVER
travel_observer.ts
Cuando ocurre un evento
del viaje, se notifica a:
• Coordinador
• Secretaria
• Transportista
end note
note right of RutaCalculadora
PATRÓN STRATEGY
route_strategy.ts
Permite cambiar el algoritmo
de cálculo de rutas:
• Más rápida
• Más segura
• Menor distancia
end note
note bottom of TransAdapter
PATRÓN ADAPTER
Adapta la interfaz del
repositorio (Target) para
que funcione con la lectura
y escritura de archivos JSON
(FuenteDatos).
end note
legend right
|= Color |= Capa |
|<#DCEEFF>| Presentación |
|<#E1F5E6>| Negocio |
|<#F1E3FA>| Datos |
|<#FFF4CC>| Entidades |
endlegend
@enduml

```

TransControl - Arquitectura de Tres Capas y Patrones de Diseño



- Tabla de responsabilidades por capa.

Capa	Clase	Responsabilidad
Presentación	GestionTransportistas View	Permite registrar, consultar, actualizar y eliminar transportistas mediante la interfaz gráfica.
Presentación	GestionViajesView	Permite gestionar viajes, visualizar información y ejecutar acciones del usuario.
Control	ControlTransportista	Coordina la lógica para crear, actualizar, eliminar y asignar transportistas.
Control	ControlViaje	Coordina la creación, consulta, cancelación y reprogramación de viajes.
Control	ControlPlanificacion	Gestiona la planificación y modificación de rutas.
Negocio	ServicioViajes	Ejecuta reglas de negocio relacionadas con viajes.
Negocio	ServicioViajeAuditado	Extiende ServicioViajes incorporando auditoría mediante Decorator.
Persistencia	RepositorioTransportistas	Almacena y recupera información de transportistas.
Persistencia	RepositorioViajes	Almacena y recupera información de viajes.
Persistencia	DatabaseConnection	Gestiona la conexión con la base de datos.
Entidad	Transportista	Representa los datos y comportamiento del transportista.
Entidad	Viaje	Representa la información operativa de un viaje.

Entidad	Ruta	Representa la ruta asignada al viaje.
Entidad	Carga	Representa la mercancía transportada.
Integración Externa	GoogleMapsAPI	Permite comunicarse con Google Maps para cálculo de rutas.

- Matriz de trazabilidad entre requisitos del CRUD y componentes implementados.

Patrón	Clase	Responsabilidad
Adapter	GoogleMap sAPI	Adaptar servicios externos de mapas al sistema interno.
Decorator	ServicioViaj eAuditado	Agregar auditoría sin modificar la lógica principal.
Observer	GestorEven tosViaje	Gestionar eventos generados por los viajes.
Observer	Notificador Console	Recibir y mostrar eventos del sistema.
Strategy	RutaMasCo rta	Implementar algoritmo de cálculo de rutas.
Strategy	EstrategiaR uta	Definir interfaz común para estrategias.



TransControl

Gestión Premium de Transporte

Iniciar Sesión

[¿Olvidaste tu contraseña?](#)

Ingresar al Sistema →

[¿No tienes cuenta? Regístrate aquí](#)

← TransControl

→



Recuperar Contraseña

Ingresar tu correo para recibir las instrucciones

Enviar Enlace →

[← Volver al Inicio de Sesión](#)



Crear Cuenta

Únete a la red TransControl

Soy un...



Conductor / Transportista



Nombres



Apellidos



domik@transcontrol.com



.....



Confirmar

Registrarme →

¿Ya tienes una cuenta? [Inicia Sesión aquí](#)



¡Hola, Juan! 🙌

ADMINISTRADOR



0

TOTAL VIAJES



0

DISPONIBLES



0

EN TRÁNSITO



0

COMPLETADOS

Acciones Rápidas



Transportistas

Gestión y control de conductores



Viajes y Rutas

Asignación y seguimiento



Documentación

Validación de licencias y SOAT



Monitoreo GPS

Rastreo de flota en tiempo real

Actividad Reciente

[Ver todos →](#)

Aún no hay viajes registrados.

Crear Transportista

Registro de Transportista

1. Datos Personales

Pedro	Freile
1719150490	0963920340
pfreile@gmail.com	
Quito	

2. Datos del Vehículo

Camión Sencillo	123 PBA-123
Toyota	2021

Cancelar

Registrar Transportista

Transportistas

Directorio de Transportistas

Gestiona los conductores y vehículos registrados

Buscar por nombre o CI...

LM	Luis Fernando Mendoza CC: 1712345678 0991234567 luis.mendoza@gmail.com	ACTIVO
CZ	Carlos Zambrano CC: 0912345678 0987654321 czambrano@hotmail.com	ACTIVO
MP	María Pérez CC: 0102345678 0976543210 mperez@yahoo.com	INACTIVO
JA	Juanito Alvarez CC: 1718230490 0963920340 jdquimbiulco@gmail.com	ACTIVO

Editar Transportista

1. Datos Personales

Luis Fernando

Mendoza

1712345678

0991234567

luis.mendoza@gmail.com

Av. Amazonas, Quito

Activo

2. Datos del Vehículo

Tipo de Vehículo...

123 Placa del Vehículo

Marca

Año

Cancelar

Guardar Cambios

Gestión de Viajes

Administra y monitorea las rutas asignadas

	Cuenca → Ambato Carga: Alimentos Perecederos 10 Ton Transportista: Sin asignar Creado: 6/17/2026	DISPONIBLE Ver Detalles
	Cuenca → Carpungo Carga: Materiales Construcción 20 Ton Transportista: Juanito Alvarez Creado: 6/17/2026	DISPONIBLE Ver Detalles



← Crear Viaje

Planificar Nuevo Viaje

Ciudad de Origen

Ej: Quito

Ciudad de Destino

Ej: Guayaquil

Tipo de Mercancía

Seleccionar...

Peso de Carga (Toneladas)

Ej: 15.5

Asignar Transportista (Opcional)

Luis Fernando Mendoza - Cédula: 1712345678 - Transportista

Observaciones Adicionales

Instrucciones especiales para el transportista...

Cancelar

✓ Crear Viaje

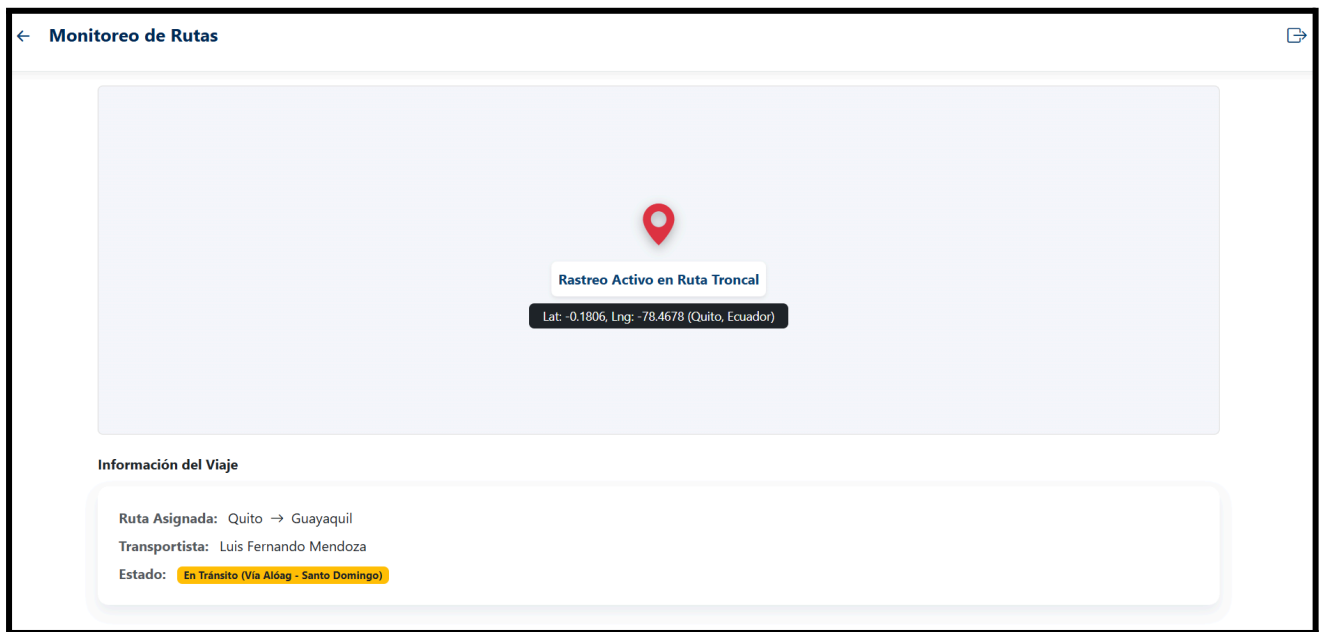
← Documentación

Control de Documentación

Auditoría y estado de requisitos legales

Subir Documento

Licencia Profesional C1 Titular: Luis Fernando Mendoza Vencimiento: 2027-12-31 VIGENTE Ver Archivo
Revisión Técnico-Mecánica Titular: Carlos Zambrano Vencimiento: 2025-06-30 VIGENTE Ver Archivo
Póliza SOAT Titular: María Pérez Vencimiento: 2024-03-15 VENCIDO Ver Archivo
Matrícula Vehicular Titular: Luis Fernando Mendoza Vencimiento: 2030-01-01 EN REVISIÓN Ver Archivo



Derivación de clases de Gestión de Transportistas y Viajes

```
@startuml DiagramaClases_TransControl

title Diagrama de Clases - Backend TransControl

skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam roundcorner 10
skinparam linetype ortho

' =====
' MODELO / ENTIDADES
' =====

package "Domain / Entities" #FFF4CC {

    class Transportista {
        + id: string
        + cedula: string
        + nombres: string
        + apellidos: string
        + correo: string
        + telefono: string
        + direccion: string
        + estado: string
    }

}
```



```

class Viaje {
    + id: string
    + origen: string
    + destino: string
    + pesoCarga: number
    + tipoMercancia: string
    + contenedor: string
    + observaciones: string
    + transportistaId: string
}
}

' =====
' INTERFACES (CONTRATOS)
' =====

package "Domain / Interfaces" #FADDF0 {

    interface ITransportistaRepository {
        + create(data: Transportista): Promise<Transportista>
        + findAll(): Promise<Transportista[]>
        + findById(id: string): Promise<Transportista>
        + update(id: string, data: Partial<Transportista>): Promise<Transportista>
        + delete(id: string): Promise<boolean>
    }

    interface IViajeRepository {
        + create(data: Viaje): Promise<Viaje>
        + findAll(): Promise<Viaje[]>
        + findById(id: string): Promise<Viaje>
        + update(id: string, data: Partial<Viaje>): Promise<Viaje>
        + delete(id: string): Promise<boolean>
    }
}

' =====
' CAPA DE DATOS (ADAPTERS)
' =====

package "Data / Adapters" #F1E3FA {

    class JsonTransportistaAdapter {
        - filePath: string
        - readData(): Transportista[]
        - writeData(data: Transportista[]): void
    }
}

```

```

        + create(data: Transportista): Promise<Transportista>
        + findAll(): Promise<Transportista[]>
        + findById(id: string): Promise<Transportista>
        + update(id: string, data: Partial<Transportista>): Promise<Transportista>
        + delete(id: string): Promise<boolean>
    }

    class JsonViajeAdapter {
        - filePath: string
        + create(data: Viaje): Promise<Viaje>
        + findAll(): Promise<Viaje[]>
        + findById(id: string): Promise<Viaje>
        + update(id: string, data: Partial<Viaje>): Promise<Viaje>
        + delete(id: string): Promise<boolean>
    }
}

' =====
'  CAPA DE NEGOCIO (SERVICES)
' =====

package "Business / Services" #E1F5E6 {

    class TransportistaService {
        - transportistaRepository: ITransportistaRepository
        + constructor(repository: ITransportistaRepository)
        + create(data: Omit<Transportista, 'id'>): Promise<Transportista>
        + getAll(): Promise<Transportista[]>
        + getById(id: string): Promise<Transportista>
        + update(id: string, data: Partial<Transportista>): Promise<Transportista>
        + delete(id: string): Promise<boolean>
    }

    class ViajeService {
        - viajeRepository: IViajeRepository
        + constructor(repository: IViajeRepository)
        + create(data: Omit<Viaje, 'id'>): Promise<Viaje>
        + getAll(): Promise<Viaje[]>
        + getById(id: string): Promise<Viaje>
    }
}

' =====
'  CAPA DE PRESENTACIÓN (CONTROLLERS)

```

```

' =====
package "Presentation / Controllers" #DCEEFF {

    class TransportistaController {
        - transportistaService: TransportistaService
        + constructor(service: TransportistaService)
        + create(req: Request, res: Response): Promise<void>
        + getAll(req: Request, res: Response): Promise<void>
        + getById(req: Request, res: Response): Promise<void>
        + update(req: Request, res: Response): Promise<void>
        + delete(req: Request, res: Response): Promise<void>
    }

    class ViajeController {
        - viajeService: ViajeService
        + constructor(service: ViajeService)
        + create(req: Request, res: Response): Promise<void>
        + getAll(req: Request, res: Response): Promise<void>
    }
}

```

```

' =====
' RELACIONES
' =====

' Implementación de las interfaces por los Adapters
JsonTransportistaAdapter ..|> ITransportistaRepository
JsonViajeAdapter ..|> IViajeRepository

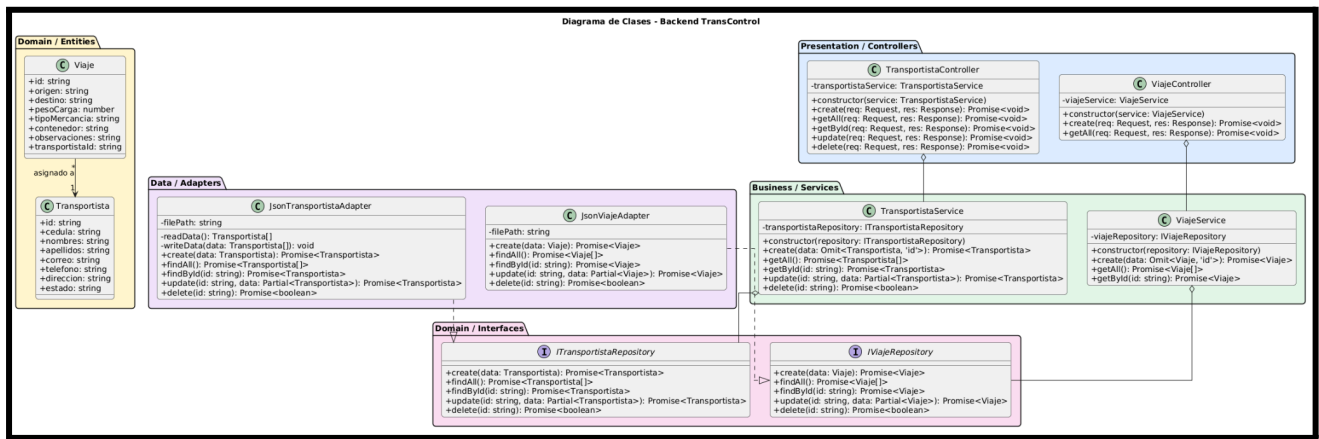
' Inyección de dependencias (Agregación)
TransportistaService o-- ITransportistaRepository
ViajeService o-- IViajeRepository

TransportistaController o-- TransportistaService
ViajeController o-- ViajeService

' Relación entre entidades
Viaje "*" --> "1" Transportista : asignado a

@enduml

```



Diagramas de secuencia de análisis de Gestión de Transportistas

```
@startuml
    Secuencia_CrearTransportista_General
```

```
title Secuencia de Análisis - Crear Transportista
```

```
skinparam shadowing false
```

```
skinparam roundcorner 10
```

```
actor "Coordinador" as Coor
```

```
boundary "PantallaGestionTransportistas" as UI
```

```
control "ControlTransportista" as Ctrl
```

```
entity "Transportista" as Trans
```

```
database "RepositorioTransportista" as Repo
```

```
Coor -> UI : Ingresa datos del transportista
```

```
Coor -> UI : Clic en botón "Crear"
```

```
UI -> Ctrl : crearTransportista(datos)
```

```
Ctrl -> Ctrl : validarDatos(datos)
```

```
Ctrl -> Repo : existeTransportista(cedula)
```

```
Repo --> Ctrl : false
```

```
Ctrl -> Trans : new Transportista(datos)
```

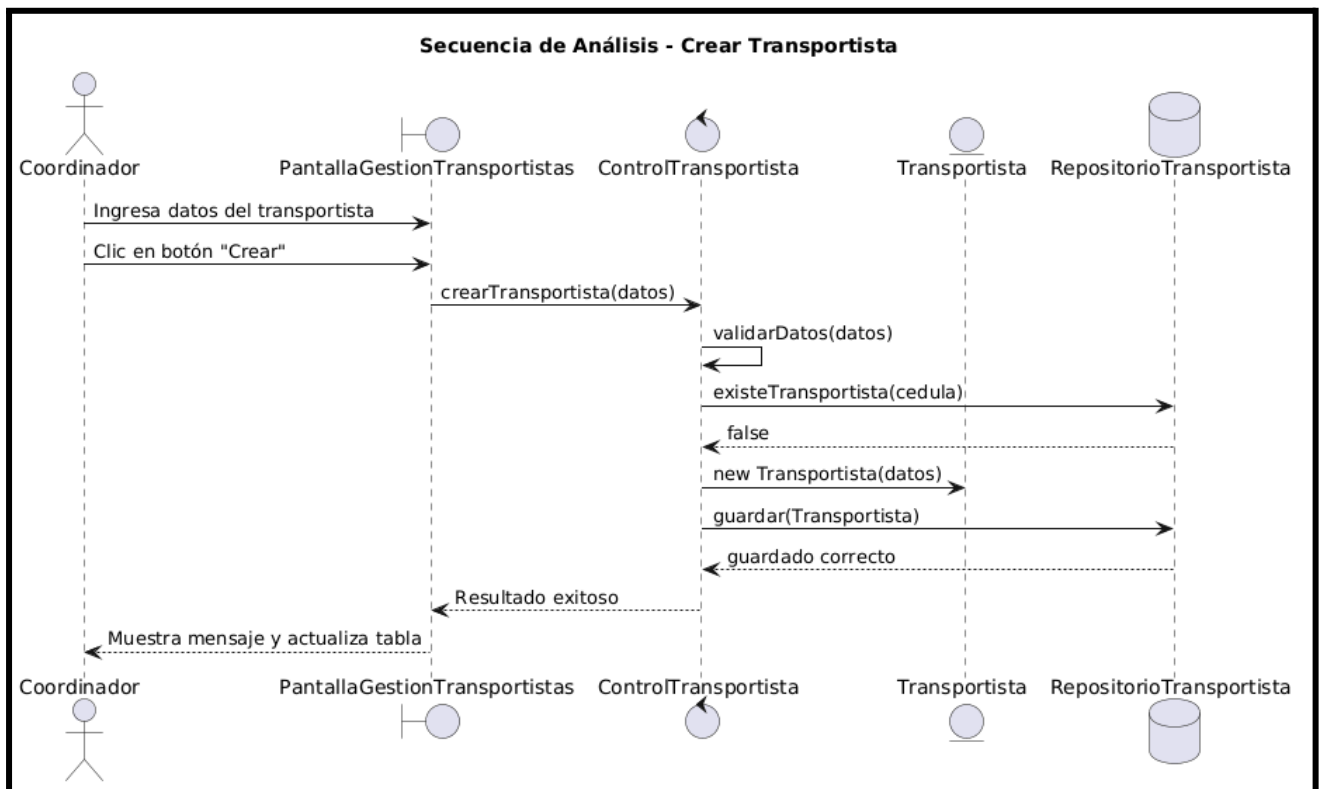
```
Ctrl -> Repo : guardar(Transportista)
```

```
Repo --> Ctrl : guardado correcto
```

```
Ctrl --> UI : Resultado exitoso
```

```
UI --> Coor : Muestra mensaje y actualiza tabla
```

```
@enduml
```



Patrones de diseño implementados en TransControl

1. Patrón Observer

1.1 ¿Qué es el patrón Observer?

Es un patrón de diseño de comportamiento que define una dependencia uno-a-muchos entre objetos, de modo que cuando un objeto (sujeto) cambia su estado, todos sus objetos dependientes (observadores) son notificados y actualizados automáticamente. Permite implementar sistemas reactivos basados en eventos donde el sujeto no necesita conocer los detalles internos de quienes lo observan.

1.2 Características

- Establece una relación uno-a-muchos con muy bajo acoplamiento entre el sujeto y sus observadores.
- El sujeto solo conoce la interfaz Observer, ignorando las clases concretas que reaccionan a los eventos.
- Los observadores pueden ser suscritos (agregados) o desuscritos (removidos) dinámicamente en tiempo de ejecución.

1.3 ¿Por qué usar Observer en la Gestión de Viajes?

El Servicio de Viajes (ViajeSubject) realiza operaciones críticas que generan eventos relevantes para todo el sistema, como la creación, inicio, finalización o cancelación de un viaje. Sin el patrón Observer, el servicio tendría que estar fuertemente acoplado para enviar correos, notificar al coordinador, y alertar al transportista cada vez que un viaje cambia. Esto violaría el Principio de Responsabilidad Única. Con Observer, el servicio de viajes simplemente "avisa" que ocurrió un evento, y cada observador reacciona de forma independiente sin alterar la lógica principal del viaje.

1.4 Mapeo del patrón Observer en el proyecto

Elemento del patrón Observer	Clase en el proyecto	Responsabilidad
Subject (Sujeto observable)	ViajeSubject	Mantiene la lista de observadores registrados y notifica a todos cuando el estado de un viaje cambia (ej: creado, en tránsito, finalizado).
Observer (Interfaz)	TravelObserver	Define el contrato <code>update(evento: String, viaje: Viaje)</code> que todos los observadores concretos están obligados a implementar.
ConcreteObserver 1	CoordinadorObserver	Reacciona a los eventos para registrar logs administrativos y monitorear alteraciones en las rutas críticas.
ConcreteObserver 2	SecretariaObserver	Reacciona para actualizar el estado del papeleo o enviar facturas cuando un viaje termina.
ConcreteObserver 3	TransportistaObserver	Recibe alertas o notificaciones push en tiempo real sobre las asignaciones de sus nuevos viajes.

2. Patrón Strategy

2.1 ¿Qué es el patrón Strategy?

Es un patrón de diseño de comportamiento que permite definir una familia de algoritmos, encapsular cada uno de ellos en una clase separada y hacer sus objetos intercambiables. El patrón permite que el algoritmo varíe independientemente de los clientes que lo utilizan.

2.2 Características

- Elimina el uso de múltiples condicionales complejos (if/else o switch) para elegir un comportamiento.
- Aísla los detalles de implementación de un algoritmo del código que lo usa.
- Permite agregar nuevas estrategias sin modificar el contexto original (Cumple el principio Abierto/Cerrado).

2.3 ¿Por qué usar Strategy en el Cálculo de Rutas?

En el sistema TransControl, al asignar un viaje, el cálculo de la ruta puede depender de diferentes factores empresariales: a veces urge llegar rápido, otras veces se transporta carga frágil y se prioriza la seguridad, o se busca ahorrar gasolina eligiendo la menor distancia. Sin Strategy, habría un bloque gigante de código evaluando qué hacer. Con Strategy, el sistema delega el cálculo a una familia de algoritmos intercambiables de forma limpia.

2.4 Mapeo del patrón Strategy en el proyecto

Elemento del patrón Strategy	Clase en el proyecto	Responsabilidad
Context (Contexto)	<code>RutaCalculadora</code>	Mantiene una referencia a la estrategia actual y permite al cliente cambiarla en tiempo de ejecución. Ejecuta la estrategia elegida.
Strategy (Interfaz)	<code>RouteStrategy</code>	Define el contrato <code>calcular(origen, destino)</code> común para todos los algoritmos de rutas soportados.
ConcreteStrategy 1	<code>RutaMasRapidaStrategy</code>	Implementa el algoritmo que prioriza vías principales y autopistas para minimizar el tiempo del viaje.
ConcreteStrategy 2	<code>RutaMasSeguraStrategy</code>	Implementa el algoritmo que evita zonas de alto riesgo, peajes peligrosos o vías en mal estado.
ConcreteStrategy 3	<code>RutaMenorDistanciaStrategy</code>	Implementa el cálculo matemático directo para encontrar la trayectoria más corta en kilómetros, ahorrando combustible.

3. Patrón Adapter

3.1 ¿Qué es el patrón Adapter?

Es un patrón de diseño estructural que permite la colaboración entre objetos con interfaces incompatibles. Funciona como un envoltorio (wrapper) que "traduce" las llamadas de una clase al formato que otra clase espera recibir.

3.2 Características

- Actúa como intermediario entre dos sistemas, permitiendo que trabajen juntos.
- Separa y oculta la complejidad de la conversión de datos.
- Permite integrar librerías externas o sistemas de almacenamiento antiguos sin reescribir la lógica de negocio actual.

3.3 ¿Por qué usar Adapter en la Persistencia de Datos?

La lógica de negocio (TransportistaService) exige trabajar con una base de datos estándar mediante métodos como `findAll` o `findById` (usando el Patrón Repositorio). Sin embargo, el proyecto guarda temporalmente los datos en archivos de texto plano (`.json`). En lugar de ensuciar la lógica de negocio enseñándole a abrir y parsear archivos de texto, el Adapter "traduce" las peticiones del servicio en operaciones de lectura y escritura de archivos del sistema operativo.

3.4 Mapeo del patrón Adapter en el proyecto

Elemento del patrón Adapter	Clase en el proyecto	Responsabilidad
Target (Interfaz objetivo)	<code>ITransportistaRepository</code>	Define la interfaz específica del dominio (los métodos CRUD estándar) que la capa de negocio espera utilizar.
Adapter (Adaptador)	<code>JsonTransportistaAdapter</code>	Es la clase que implementa la interfaz Target. Convierte las llamadas CRUD en lecturas y escrituras del módulo nativo <code>fs</code> de Node.js.
Adaptee (Adaptado)	<code>JsonStorage</code> (Archivo JSON)	Es el sistema o librería incompatible. En este caso, representa el archivo físico <code>transportistas.json</code> que solo entiende texto plano.

Client (Cliente)	TransportistaService	Usa el sistema llamando a los métodos de la interfaz Target, ignorando por completo que por debajo se manipulan archivos JSON.
------------------	----------------------	--

CONCLUSIONES

- Consideramos que la implementación de la arquitectura de tres capas fue fundamental para organizar adecuadamente el sistema, ya que permitió separar las responsabilidades entre las capas de Datos, Lógica de Negocio y Presentación, facilitando el mantenimiento y comprensión del proyecto.
- Observamos que el patrón Adapter resolvió de forma efectiva el problema de compatibilidad entre formatos de datos externos y el modelo interno del sistema. Gracias a este patrón fue posible integrar información proveniente de diferentes fuentes sin modificar la estructura principal del CRUD.
- Creemos que el patrón Decorator aportó una solución flexible para incorporar funcionalidades adicionales, como auditoría y validaciones, evitando cambios directos en las clases base y favoreciendo la extensibilidad del sistema.

RECOMENDACIONES

- Recomendamos mantener la separación de responsabilidades entre capas para conservar una arquitectura organizada y facilitar futuras modificaciones o ampliaciones del sistema.
- Sugerimos continuar utilizando el patrón Adapter para integrar nuevas fuentes de información externas, evitando cambios innecesarios en las clases internas del proyecto.
- Consideramos conveniente emplear Decorator para agregar futuras funcionalidades transversales, como generación de reportes, control de acceso o registro avanzado de actividades.
- Recomendamos gestionar adecuadamente el registro y eliminación de observadores dentro del patrón Observer para evitar referencias innecesarias y garantizar un comportamiento eficiente del sistema.